

Docker & DevOps: The Complete Guide

Condensed Course Notes — Building, Shipping & Deploying Containerized Applications

Course Notes · 2026

Contents

Chapter 1: Introduction to DevOps and Containerization	3
Chapter 2: Docker Architecture	6
Chapter 3: Installing Docker and Getting Started	8
Chapter 4: The Docker Command-Line Toolkit	10
Chapter 5: Environment Variables and Container Configuration	12
Chapter 6: Docker Images and the Dockerfile	14
Chapter 7: Docker Networking	17
Chapter 8: Docker Storage — Volumes, Bind Mounts, and tmpfs	19
Chapter 9: Docker Compose — Multi-Container Applications	21
Chapter 9b: Advanced Docker Compose Features	24
Chapter 10: Docker Registries and Image Distribution	26
Chapter 11: The Docker Engine, Deep Dive	28
Chapter 11b: Beyond Docker — The Broader Container Ecosystem	30
Chapter 12: Docker on Windows and macOS	31
Chapter 13: Container Orchestration — Why and What	32
Chapter 14: Docker Swarm	33
Chapter 15: Kubernetes Fundamentals	36
Chapter 16: CI/CD and Deploying Containerized Applications	40
Chapter 16b: Capstone Project — Containerizing and Deploying a Three-Tier Application	42
Chapter 17: Best Practices, Security, and Common Pitfalls	45
Chapter 17b: Troubleshooting Docker — A Practical Field Guide	48
Chapter 18: Conclusion and Next Steps	49
Appendix A: Complete Command Cheat Sheet	50
Appendix B: Dockerfile Instruction Reference	51
Appendix C: Glossary	52
Appendix D: Review Questions and Answers	53
Appendix E: Further Resources	55

Chapter 1: Introduction to DevOps and Containerization

1.1 What Is DevOps?

DevOps is a set of cultural philosophies, practices, and tools that raises an organization's ability to deliver software at high velocity. It merges Development and IT Operations, historically siloed teams with conflicting incentives (developers want to ship features fast; operations wants stability, which traditionally meant resisting change). DevOps makes one team, or tightly collaborating teams, responsible for the whole lifecycle: plan, code, build, test, release, deploy, operate, monitor.

1.1.1 Core DevOps Principles

- **Automation** — builds, tests, deployments, and provisioning become scripted, repeatable pipelines.
- **Continuous Integration (CI)** — code is merged into a shared repository often, and every merge is built and tested automatically.
- **Continuous Delivery / Deployment (CD)** — validated code is prepared for release (Delivery) or pushed straight to production (Deployment).
- **Infrastructure as Code (IaC)** — servers, networks, and load balancers are defined in version-controlled files, not set up by hand.
- **Monitoring and Observability** — logs, metrics, and traces let teams detect and diagnose problems quickly.
- **Shared ownership** — "you build it, you run it."

1.1.2 The DevOps Toolchain

Phase	Example Tools
Plan	Jira, Trello, Azure Boards
Code	Git, GitHub, GitLab, Bitbucket
Build	Maven, Gradle, npm, Docker
Test	JUnit, Selenium, pytest
Release/Deploy	Jenkins, GitHub Actions, GitLab CI, Azure DevOps, ArgoCD
Operate	Kubernetes, Ansible, Terraform
Monitor	Prometheus, Grafana, ELK Stack, Datadog

Docker sits at the heart of Build, Release, and Operate: it standardizes how software is packaged and how it runs.

1.2 The Problem Docker Solves

Traditionally, deploying meant provisioning a machine, installing an OS, then manually installing the runtime (a specific Node.js, Python, or Java version), system libraries, and finally the application. This produced the famous "**it works on my machine**" problem: the app works on the developer's laptop, behaves differently or crashes on a test server with slightly different dependencies, and the mismatch repeats between test and production and across production servers.

Older remedies were heavyweight. *Setup documentation* is brittle, goes stale, and needs manual execution. *Virtual machines* bundle a full guest OS and guarantee consistency, but at a steep cost in disk, memory, and boot time.

1.3 Virtual Machines vs. Containers

1.3.1 Virtual Machine Architecture

A VM emulates a whole computer. From the bottom up: physical hardware, (optional) host OS, a **hypervisor** (VMware ESXi, VirtualBox, Hyper-V, KVM) that allocates resources, a full **guest OS** per VM with its own kernel and drivers, then the application. Images are often several gigabytes and boot takes tens of seconds to minutes.

1.3.2 Container Architecture

A container bundles no guest OS. Containers **share the host kernel** and isolate processes with kernel features. The stack is hardware, a single host OS (typically Linux), a container runtime (Docker Engine, containerd), and containers holding only the app and the libraries it needs. Images are megabytes, and startup takes a fraction of a second to a few seconds because there is no OS boot.

1.3.3 Side-by-Side Comparison

Aspect	Virtual Machines	Containers
Isolation	Hardware-level virtualization; separate kernels	OS-level (process) virtualization; shared kernel
Bundled	Guest OS + app + dependencies	App + dependencies only
Image size	Gigabytes	Megabytes
Startup	Seconds to minutes	Milliseconds to seconds
Overhead / density	High / fewer per host	Near-native / many per host
Portability	Portable but heavy	Highly portable and light
Security boundary	Very strong	Weaker by default, strong when configured properly

1.3.4 Why the Shared Kernel Matters

A Linux container image only runs on a compatible Linux kernel, which is why Docker on Windows/Mac runs a lightweight Linux VM underneath (Chapter 12). Containers trade total isolation for much lower overhead and faster startup. For most workloads (web servers, APIs, workers, databases) namespaces and cgroups provide enough isolation; for untrusted multi-tenant code, VMs or micro-VMs such as Firecracker remain relevant.

1.4 What Exactly Is a Container?

A container is a standardized, self-contained unit of software that packages code with its libraries, binaries, configuration, and dependencies so it behaves identically on a laptop, test server, or cloud cluster. Two ideas hide in the word:

1. **Container image** — an immutable, read-only template: a filesystem snapshot plus metadata (default command, environment variables, exposed ports). Built once, run many times.
2. **Container** — a running or stopped *instance* of an image, like an object is an instance of a class. Many containers can start from one image.

Two Linux kernel features make this work:

- **Namespaces** provide isolation. Each container gets its own view of process IDs (PID), network interfaces (net), mounts (mnt), hostname (UTS), IPC, and user IDs (user).
- **Control groups (cgroups)** limit and account for CPU, memory, disk I/O, and network bandwidth so one container cannot starve the others.

Docker did not invent either (LXC/LXD used them earlier). Its innovation was a friendly toolchain, a standard image format (later the **OCI** specifications), and a registry ecosystem (Docker Hub), which drove mainstream adoption from about 2013.

1.5 Benefits of Containerization

- **Consistency** — build once, run anywhere; the same image runs on laptop, CI, and production.
- **Isolation** — separate filesystem, processes, and network stack prevent dependency conflicts.
- **Lightweight and fast** — higher density and quick startup, essential for auto-scaling.
- **Versioned environments** — a Dockerfile lives in source control next to the code.
- **Microservices enablement** and **efficient resource use** compared with many VMs.
- **Simpler CI/CD** — the image that passes tests is the exact artifact promoted to production, eliminating environment drift.

1.6 A Brief History and the Role of Docker Inc.

- **2008** — Linux gains full cgroups support; LXC emerges as an early container manager.
- **2013** — Docker, Inc. (originally dotCloud) releases Docker, first on LXC, then its own libcontainer. Its contribution was developer experience: simple CLI, standard image format, layered storage, Docker Hub.

- **2015** — The Open Container Initiative (OCI) is founded to standardize image formats and runtimes.
- **2017 onward** — Docker is split into composable pieces: **containerd** (donated to the CNCF) and **runc** (low-level OCI runtime), now used directly by Kubernetes too.
- **Today** — "Docker" means both Docker Inc.'s products (Desktop, Engine, Hub) and, loosely, the OCI ecosystem. Kubernetes dominates orchestration, often using containerd directly (Chapter 11b).

1.7 What This Course Covers

Installing Docker and running containers; the full CLI; Dockerfiles and multi-stage builds; networking; volumes and bind mounts; Docker Compose; registries; Engine internals; Windows/macOS specifics; Docker Swarm; Kubernetes fundamentals; and CI/CD and production deployment.

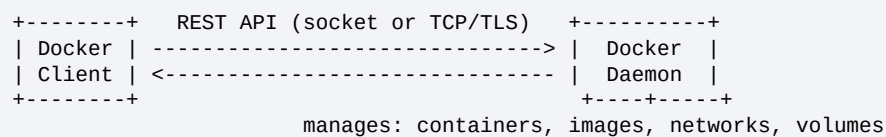
Chapter 2: Docker Architecture

2.1 Client-Server Architecture

Docker has three major components:

1. **Docker Client** (**docker**) — the CLI (or GUI such as Docker Desktop) you type commands into.
2. **Docker Daemon** (**dockerd**) — a background service that builds images, runs containers, and manages networks and volumes; it listens for API requests.
3. **Docker Registry** — storage and distribution for images. Docker Hub is the default public registry; organizations often run private ones (Chapter 10).

Client and daemon talk over a REST API, either locally through a Unix socket (`/var/run/docker.sock`, the default) or remotely over TCP, optionally secured with TLS. That is why you can run **docker** on a laptop against a remote server, and why Docker Desktop's client talks to a daemon in a hidden Linux VM.



2.2 The Docker Daemon's Responsibilities

The daemon owns the lifecycle of Docker objects: building images, pulling and pushing to registries, creating/starting/stopping/removing containers, managing networks and volumes, and exposing the REST API. It can also coordinate with other daemons for Swarm services (Chapter 14).

2.3 Docker Objects

Object	Description
Image	Read-only template for creating a container; built in layers
Container	Runnable instance of an image; can be started, stopped, moved, deleted
Network	Communication between containers and with the outside world
Volume	Persistent data that outlives the container
Plugin	Extends Docker (third-party volume or network drivers)

2.4 Underlying Technology: containerd and runc

dockerd does not create containers itself; there is a chain of responsibility:

```
docker (CLI) -> dockerd (images, networks, volumes, build, API)
               -> containerd (container lifecycle, image transfer/storage)
               -> containerd-shim (lets containerd restart without killing containers)
               -> runc (creates namespaces/cgroups, starts the process)
```

- **containerd** was donated to the CNCF in 2017 and is a graduated project used by Docker and by Kubernetes (via the Container Runtime Interface, CRI).
- **runc** implements the OCI Runtime Specification, the industry-standard contract for "how to start a container," derived from Docker's libcontainer.

So "Docker" is a friendly toolkit over standardized, swappable parts, which is why Kubernetes can run containers without the Docker Engine (using containerd or CRI-O).

2.5 Images Are Built From Layers

Images are stacked **read-only layers**, typically one per Dockerfile instruction. Layers are cached and content-addressed (by hash). Images sharing an ancestor (both **FROM ubuntu:22.04**) store that layer once. On rebuild, unchanged layers come from cache and only layers after the first changed instruction are rebuilt.

A running container adds one thin **writable layer** on top, using **copy-on-write**: modifying a file from a read-only layer first copies it up, leaving the image untouched.

```
+-----+
| Writable container layer | per container; gone when removed
+-----+
| COPY app code           | read-only, shared by every
| RUN pip install ...     | container using this image
| FROM python:3.12        |
+-----+
```

Starting a container only needs a new empty writable layer, which is why it is so fast. It also means **data written to the container layer is lost when the container is removed** unless persisted with volumes or bind mounts (Chapter 8).

2.6 Docker Desktop vs. Docker Engine

- **Docker Engine** — the open-source client-server core (**dockerd**, CLI, API). Native to Linux.
- **Docker Desktop** — a commercial app for Windows/macOS (free for personal use and small business) bundling a Linux VM for the Engine, the CLI, Compose, optional single-node Kubernetes, and a GUI.
- **Docker Hub** — the default public registry and cloud service.
- **Docker Compose** — tool and file format for multi-container apps (Chapter 9).

Chapter 3: Installing Docker and Getting Started

3.1 Installation Options Overview

Platform	Recommended Install	Notes
Linux	Docker Engine (native)	Runs natively; no VM needed
macOS	Docker Desktop	Runs a lightweight Linux VM
Windows	Docker Desktop (WSL2 backend)	Uses the WSL2 Linux kernel
Cloud servers	Engine via script or package manager	e.g., AWS EC2, DigitalOcean

3.2 Installing on Linux (Ubuntu example)

```
# 1. Remove old versions
sudo apt-get remove docker docker-engine docker.io containerd runc
# 2. Add Docker's official apt repository
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
# 3. Install Engine, CLI, containerd, plugins
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
# 4. Verify
sudo docker run hello-world
```

3.2.1 Running Docker Without sudo

The daemon socket is owned by root, so add your user to the **docker** group:

```
sudo groupadd docker      # create group if missing
sudo usermod -aG docker $USER # add current user
newgrp docker             # activate in this shell
docker run hello-world    # now works without sudo
```

Security note: docker group membership is effectively root on the host (a container can bind-mount the whole host filesystem). Add only trusted users.

3.2.2 Enabling Docker to Start on Boot

```
sudo systemctl enable docker.service
sudo systemctl enable containerd.service
sudo systemctl status docker
```

3.3 Installing Docker Desktop on macOS

Download the installer (Intel or Apple Silicon), drag Docker.app to /Applications, launch it and grant permission for helper components, then verify with `docker --version` and `docker run hello-world`. Desktop runs a purpose-built Linux VM (formerly HyperKit, now Apple's Virtualization.framework) because macOS's XNU kernel cannot run Linux containers natively.

3.4 Installing Docker Desktop on Windows

Enable WSL2 with `wsl --install`, run the Docker Desktop installer with "Use WSL 2 instead of Hyper-V" selected (the default on modern Windows 10/11), restart, then verify from PowerShell or a WSL2 terminal. Containers run on WSL2's real Linux kernel, giving near-native performance and beating the older, heavier Hyper-V backend.

3.5 Verifying and Exploring Your Installation

```
docker version      # client and daemon versions
docker info         # containers, images, storage driver, cgroup version
docker system df    # disk usage by images, containers, volumes
```

`docker run hello-world` is the canonical smoke test: Docker looks for the image locally, pulls it from Docker Hub, creates and runs a container that prints a message, and the container exits.

3.6 Docker Desktop GUI Overview

The dashboard lets you browse and start/stop/delete containers, view logs, inspect images and volumes, open a terminal in a container, set CPU/memory/disk limits for the VM, and enable a single-node Kubernetes cluster. The GUI is convenient for exploring, but automation and production rely on the CLI.

3.7 Docker Playground / Labs

Play with Docker (labs.play-with-docker.com) offers free, temporary, browser-based Docker environments for labs, multi-node Swarm tests, and safely trying destructive commands.

Chapter 4: The Docker Command-Line Toolkit

The practical core of day-to-day container management. Commands are grouped by function.

4.1 docker run — Creating and Starting Containers

docker run is the most-used command; it combines **docker create** and **docker start**.

```
docker run [OPTIONS] IMAGE[:TAG] [COMMAND] [ARG...]  
docker run ubuntu          # run default command, then exit  
docker run ubuntu sleep 5  # run a command, then exit  
docker run -it ubuntu bash  # interactive shell with pseudo-TTY  
docker run -d nginx        # detached (background)
```

4.1.2 Key Flags for docker run

Flag	Purpose
-d, --detach	Run in the background, print container ID
-it	-i keeps STDIN open, -t allocates a TTY; for interactive shells
--name NAME	Human-readable name instead of a random one
-p HOST:CONTAINER	Publish a container port on a host port
-P	Publish all exposed ports to random host ports
-e KEY=VALUE	Set an environment variable
--env-file FILE	Load environment variables from a file
-v HOST:CONTAINER	Bind or volume mount (Chapter 8)
--rm	Remove the container automatically on exit
--network NAME	Attach to a specific network
--restart POLICY	no, on-failure[:max], always, unless-stopped
-m, --cpus	Limit memory (-m 512m) and CPU (--cpus="1.5")
-w, -u	Working directory; run as a given user/UID instead of root
--entrypoint	Override the image's ENTRYPOINT

4.1.3 Worked Example: Nginx Web Server

```
docker run -d --name webserver -p 8080:80 nginx
```

Pulls nginx if needed, starts it detached, names it **webserver**, and maps host port 8080 to container port 80. Visit <http://localhost:8080> for the welcome page.

4.2 Inspecting Container State

```

docker ps                # running containers
docker ps -a             # including stopped ones
docker ps -q             # IDs only (scripting)
docker ps --filter "status=exited"
docker inspect CONTAINER # full JSON: config, network, mounts, env
docker inspect --format '{{.NetworkSettings.IPAddress}}' CONTAINER
docker top CONTAINER     # processes inside
docker stats [CONTAINER] # live CPU/memory/network

```

4.3 Container Lifecycle Commands

```

docker start CONTAINER # start a stopped container
docker stop CONTAINER  # SIGTERM, then SIGKILL after 10s
docker stop -t 30 CONTAINER # custom grace period
docker restart CONTAINER
docker pause / unpause CONTAINER # freeze/resume all processes
docker kill CONTAINER           # SIGKILL immediately
docker rm CONTAINER             # remove a stopped container
docker rm -f CONTAINER          # force-remove a running one
docker rm $(docker ps -aq)      # remove ALL (add -f for running)

```

4.4 Logs and Debugging

```

docker logs CONTAINER # all stdout/stderr so far
docker logs -f CONTAINER # follow, like tail -f
docker logs --tail 100 CONTAINER
docker logs --since 10m CONTAINER

```

4.4.1 docker exec — Running Commands in a Running Container

```

docker exec CONTAINER COMMAND # one-off command
docker exec -it CONTAINER bash # interactive shell (use sh on Alpine)

```

run creates a brand-new container from an image; **exec** starts an extra process inside an already-running one.

4.5 Copying Files Between Host and Container

```

docker cp CONTAINER:/path/in/container ./local/path # container -> host
docker cp ./local/file CONTAINER:/path/in/container # host -> container

```

4.6 Working with Images

```

docker images | docker image ls # list local images
docker pull / push IMAGE[:TAG] # download / upload
docker rmi IMAGE                 # remove (also: docker image rm)
docker image prune [-a]          # remove dangling (or all unused) images
docker history IMAGE             # layer history
docker tag SOURCE TARGET         # new tag for an existing image

```

4.7 System Cleanup

Stopped containers, dangling images, unused networks and volumes accumulate and use disk.

```

docker system df          # disk usage
docker system prune       # stopped containers, dangling images, unused networks
docker system prune -a    # also all unused images
docker system prune -a --volumes # also unused volumes: DELETES DATA
docker container prune | docker volume prune | docker network prune

```

4.8 Hands-On Lab: Container Basics

```

docker run -it ubuntu bash          # 1. explore
cat /etc/os-release ; exit
docker run -d --name mynginx -p 8081:80 nginx # 2. background
docker ps ; curl http://localhost:8081
docker logs mynginx                 # 3. logs and exec
docker exec -it mynginx bash        # ls /usr/share/nginx/html ; exit
docker stop mynginx ; docker ps -a  # 4. stop, restart, clean up
docker start mynginx ; docker rm -f mynginx

```

4.9 Common Pitfalls

- Forgetting **-it**: without a TTY and open STDIN an interactive shell exits immediately.
- Confusing **stop** and **rm**: **stop** keeps the container (and its unpersisted data) and its disk usage; **rm** deletes it.
- **Port already in use**: **-p 8080:80** fails if the host port is bound by another process or container; pick another.
- **Name conflicts**: **--name web** fails if any container, even stopped, already has that name; **docker rm web** first.

Chapter 5: Environment Variables and Container Configuration

5.1 Why Environment Variables Matter

Containers should be configurable without rebuilding the image. The Twelve-Factor App methodology says configuration (database URLs, credentials, feature flags) belongs in the environment, not in code. The same image can then move from development to staging to production with only the variables changing.

5.2 Setting Environment Variables at Runtime

```

docker run -e APP_ENV=production myapp
docker run -e DB_HOST=db.example.com -e DB_PORT=5432 myapp

```

5.2.1 Loading from a File

```

# .env
DB_HOST=db.example.com
DB_PORT=5432
DB_PASSWORD=supersecret

docker run --env-file .env myapp

```

Security note: `.env` files hold secrets; never commit them (use `.gitignore`). In production prefer a secrets manager (Docker Secrets, Vault, AWS Secrets Manager, Kubernetes Secrets), because environment variables are visible through `docker inspect` and process listings.

5.3 Setting Default Environment Variables in a Dockerfile

```
FROM node:20
ENV NODE_ENV=production
ENV PORT=3000
```

Any `-e` value at `docker run` overrides the Dockerfile's `ENV`: image-level defaults, overridden at runtime.

5.4 Viewing a Container's Environment Variables

```
docker exec CONTAINER env
docker inspect CONTAINER --format='{{range .Config.Env}}{{println .}}{{end}}'
```

5.5 Worked Example: A Configurable Database Container

```
docker run -d --name mydb \
-e MYSQL_ROOT_PASSWORD=rootpass -e MYSQL_DATABASE=appdb \
-e MYSQL_USER=appuser -e MYSQL_PASSWORD=apppass \
-p 3306:3306 mysql:8
```

The image's entrypoint script reads these variables and initializes the database, with no config-file editing. Official images for Postgres, MongoDB, Redis, and RabbitMQ use the same pattern.

5.6 Environment Variables in Docker Compose

```
services:
  web:
    image: myapp
    environment:
      - APP_ENV=production
      - DB_HOST=db
    env_file:
      - .env
```

5.7 Best Practices

- Keep secrets out of Dockerfiles: `ENV` values are baked into the image and visible via `docker history`.
- Use `.env` for local development, a real secrets manager in production.
- Document required variables (README or `.env.example`).
- Fail fast: validate required variables at startup and exit with a clear error.

Chapter 6: Docker Images and the Dockerfile

6.1 What Is an Image, Precisely?

An image is an immutable, layered filesystem snapshot plus metadata (default command, exposed ports, working directory, default environment). Images have a name and tag (`nginx:1.25`, `python:3.12-slim`) and a content-addressable **digest** (SHA-256 of the manifest); identical digests mean byte-for-byte identical images whatever the tags.

With no tag, Docker uses `latest`, which is only a convention, not a guarantee of newest. Avoid `latest` in production; pin a version tag or, better, a digest.

6.2 The Dockerfile

A plain-text file of instructions the build engine runs in order; each typically creates a layer.

6.2.1 Core Instructions

```
FROM python:3.12-slim      # base image; always first (ARG may precede)
LABEL maintainer="team@example.com"
WORKDIR /app              # working dir for later instructions and runtime
COPY requirements.txt .    # copy from build context into the image
ADD archive.tar.gz /app/  # COPY + tar extraction + remote URLs (prefer COPY)
RUN pip install --no-cache-dir -r requirements.txt  # BUILD-time command, new layer
COPY . .
ENV PORT=8000             # default environment variable
EXPOSE 8000               # documents the port; does NOT publish it (-p does)
USER appuser              # run as non-root (security best practice)
ENTRYPOINT ["python", "app.py"]  # fixed command run at start
CMD ["--port", "8000"]    # default args, overridable at docker run
```

6.2.2 Full Instruction Reference

Instruction	Purpose
FROM, LABEL	Set the base image; add key-value metadata
RUN	Execute a command at build time, creating a layer
COPY, ADD	Copy from build context (ADD also extracts tar and fetches URLs)
WORKDIR, ENV	Set working directory; set persistent environment variable
ARG	Build-time-only variable, absent from the final container
EXPOSE, VOLUME	Document listening port; declare a mount point for external data
USER	User (and group) for later instructions and runtime
ENTRYPOINT, CMD	Fixed primary command; default arguments or default command
HEALTHCHECK	How Docker tests container health
STOPSIGNAL	Signal used to stop the container
ONBUILD	Defer an instruction until this image is used as a base

6.3 Building an Image

```
docker build -t myapp:1.0 .
```

`-t` names and tags the image; `.` is the **build context**, the directory sent to the daemon and available to COPY/ADD, so keep it lean.

```
docker build -f Dockerfile.prod -t myapp:prod .    # alternate Dockerfile
docker build --no-cache -t myapp:1.0 .            # ignore layer cache
docker build --build-arg VERSION=1.2.3 -t myapp . # pass build ARG
docker build --progress=plain .                   # verbose output
```

6.4 The .dockerignore File

Like `.gitignore`, it excludes files from the build context: `.git`, `node_modules`, `*.log`, `.env`, `Dockerfile`, `README.md`. It speeds up builds and keeps secrets and bloat out of images.

6.5 Layer Caching in Depth

The cache works top to bottom: once one instruction's inputs change, every later instruction rebuilds. Order therefore matters.

```
# Anti-pattern: npm install reruns on ANY code change
COPY . .
RUN npm install

# Best practice: copy manifests first, install, then copy source
COPY package*.json ./
RUN npm install
COPY . .
```

Installing dependencies before copying source is one of the highest-leverage optimizations, since dependencies change rarely and install slowly while source changes constantly.

6.6 Multi-Stage Builds

Multiple **FROM** stages let later stages copy only chosen artifacts from earlier ones, leaving build tools and source behind.

```
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # outputs to /app/dist

FROM nginx:1.25-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Benefits: **smaller images** (over a gigabyte down to tens of MB), **better security** (smaller attack surface), and **simpler pipelines** (one Dockerfile captures build and package). It is the standard approach for Go, Java, Rust, C++ and frontend frameworks (React, Vue, Angular).

6.7 Choosing a Base Image

Style	Example	Trade-off
Full OS	ubuntu:22.04 (~70MB+)	Familiar tooling, larger
Slim	python:3.12-slim	Smaller, fewer preinstalled packages
Alpine	node:20-alpine (~5-10MB)	Very small; musl libc can cause subtle compatibility issues
Distroless	gcr.io/distroless/*	No shell or package manager: most secure, harder to debug
scratch	FROM scratch (~0MB)	Empty; only for static binaries (e.g., Go)

6.8 CMD vs ENTRYPOINT

One of the most misunderstood parts of Docker.

Two forms. *Exec form* `CMD ["executable", "arg"]` (preferred) runs the process directly as PID 1, so it receives signals such as SIGTERM for graceful shutdown. *Shell form* `CMD command arg` wraps it in `/bin/sh -c`, making the shell PID 1, which can break signal handling.

```
# CMD alone: default, fully replaceable
FROM ubuntu
CMD ["echo", "Hello, World"]
# docker run myimage          -> Hello, World
# docker run myimage echo Hi   -> Hi      (CMD replaced)

# ENTRYPOINT alone: args are appended
ENTRYPOINT ["echo"]
# docker run myimage Hello     -> echo Hello
# docker run --entrypoint ls myimage -la /   (explicit override)

# Recommended: both
ENTRYPOINT ["echo"]
CMD ["Hello, World"]
# docker run myimage          -> echo Hello, World
# docker run myimage "Custom" -> echo Custom
```

The official `postgres` image works this way: `ENTRYPOINT ["docker-entrypoint.sh"]` with `CMD ["postgres"]`, so initialization always runs while the final command can be overridden.

Form	Behaviour with <code>docker run ARGS</code>	Typical use
CMD only	Fully replaced	Simple default command
ENTRYPOINT only	Arguments appended; executable fixed	Force a specific executable
ENTRYPOINT + CMD	CMD gives overridable default args	Most flexible; CLI-like images

6.9 HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1
```

Docker runs the check periodically; `docker ps` shows healthy, unhealthy, or starting.

Orchestrators (Swarm, Kubernetes probes) use health to route traffic or restart containers.

6.10 Image Naming, Tagging, and Versioning

```
docker tag myapp:1.0 myrepo/myapp:1.0
docker tag myapp:1.0 myrepo/myapp:latest
# Reference form: [REGISTRY_HOST[:PORT]/]REPOSITORY[:TAG]
```

Docker Hub is assumed when the registry host is omitted. Common tags: semantic versions (**1.4.2**), **latest** for the newest stable build, and git short-SHA tags (**a1b2c3d**) for immutable traceability to a source commit, a very common CI/CD pattern.

Chapter 7: Docker Networking

7.1 Why Networking Matters

Real applications are rarely a single container: a web server must reach a database, a cache, and other services, and be reachable from outside. Docker networking controls how containers talk to each other, the host, and the world.

7.2 Network Drivers

Driver	Scope	Typical Use
bridge	Single host	Default; isolated private network for containers on one host
host	Single host	Shares the host's network stack; no isolation, no port mapping
none	Single host	No networking; total isolation
overlay	Multi-host	Connects containers across Docker hosts (Swarm, Chapter 14)
macvlan	Single host	Gives a container its own MAC address so it looks like a physical device
ipvlan	Single host	Like macvlan but shares one MAC address

7.3 The Default Bridge Network

Docker creates a network named **bridge**; containers started without **--network** join it.

```
docker network ls
docker network inspect bridge    # subnet, gateway, connected containers
```

- Containers get IPs on a private subnet (commonly 172.17.0.0/16).
- They reach each other by IP but **not by name**: there is no DNS on the default bridge, a common beginner trap.
- They reach the internet through NAT via the host, and are reachable from outside only through **-p**.

7.4 User-Defined Bridge Networks (Recommended)

```
docker network create mynetwork
docker run -d --name db --network mynetwork postgres
docker run -d --name web --network mynetwork myapp
```

- An embedded DNS server lets containers resolve each other by name (**web** connects to **db** simply as **db**).
- Only containers on the same network can talk by default, isolating unrelated stacks.
- Containers can join or leave networks while running: **docker network connect NETWORK CONTAINER**.

Compose relies on this: it creates a user-defined network per project (Chapter 9).

7.5 The host Network

docker run -d --network host nginx shares the host's network namespace. There is no NAT overhead and no need for **-p**, giving the best performance, but no isolation and possible port conflicts. It is Linux-only and behaves differently or is unsupported on Docker Desktop for Mac/Windows because of the VM.

7.6 The none Network

docker run -d --network none myapp gives only a loopback interface. Useful for batch jobs that need only disk and CPU, or maximal isolation.

7.7 Publishing Ports

```
docker run -p 8080:80 nginx           # host 8080 -> container 80
docker run -p 127.0.0.1:8080:80 nginx # bind to localhost only
docker run -p 8080:80 -p 8443:443 nginx # several ports
docker run -P nginx                   # all EXPOSEd ports -> random host ports
```

On Linux, publishing works through iptables NAT rules that Docker manages.

7.8 Inspecting Network Configuration

```
docker network ls | inspect NETWORK
docker network create --driver bridge --subnet 172.20.0.0/16 mynet
docker network connect | disconnect NETWORK CONTAINER
docker network rm NETWORK | docker network prune
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' CONTAINER
```

7.9 Container-to-Container Communication: Worked Example

```
docker network create appnet
docker run -d --name db --network appnet -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --name api --network appnet \
  -e DATABASE_URL=postgres://postgres:secret@db:5432/postgres myapi
# "api" resolves the hostname "db" via the network's embedded DNS
```

7.10 DNS and Service Discovery Summary

Network type	Container-name DNS?
Default bridge	No
User-defined bridge	Yes
host	N/A (uses the host's network stack and DNS)
Swarm overlay	Yes, plus built-in load balancing across replicas

7.11 Common Pitfalls

- Expecting name resolution on the default bridge; use a user-defined network (or the legacy, discouraged `--link`, or IPs).
- Assuming `EXPOSE` publishes a port. It is documentation only; use `-p/-P` or Compose `ports:`.
- Apps listening only on `127.0.0.1` inside a container are unreachable from outside even with correct publishing; bind to `0.0.0.0`.
- Host port conflicts when several containers map to the same host port.

Chapter 8: Docker Storage — Volumes, Bind Mounts, and tmpfs

8.1 Why Persistent Storage Is Necessary

A container's writable layer is deleted when the container is removed. That is fine for stateless apps, but databases, uploads, and logs that must survive redeployments need one of three storage mechanisms.

8.2 The Three Storage Mechanisms

Mechanism	Managed by	Location	Typical use
Volumes	Docker	Docker area (<code>/var/lib/docker/volumes/...</code> on Linux)	Recommended default for persistent application data
Bind mounts	You	Any host path	Source code in development; sharing specific host files
tmpfs	Docker (memory)	RAM only, never on disk	Sensitive temporary data; Linux only

8.3 Volumes

Volumes are fully Docker-managed, so they are portable, easy to back up, independent of host directory layout, and work the same on Linux, Mac, and Windows.

8.3.1 Creating and Using Volumes

```
docker volume create mydata
docker volume ls | inspect mydata | rm mydata | prune
docker run -d --name db -v mydata:/var/lib/postgresql/data postgres:16
```

mydata is created automatically on first use and persists, with its data, after the container is removed until you delete it.

8.3.2 Anonymous Volumes

`docker run -d -v /var/lib/postgresql/data postgres:16` creates a volume with a random name. These are easy to lose track of, so prefer named volumes for anything that must persist.

8.3.3 Sharing a Volume Across Containers

```
docker volume create shared_data
docker run -d --name writer -v shared_data:/data busybox \
  sh -c "while true; do date >> /data/log.txt; sleep 1; done"
docker run -d --name reader -v shared_data:/data:ro busybox tail -f /data/log.txt
```

The **:ro** suffix mounts read-only, a least-privilege pattern.

8.4 Bind Mounts

Bind mounts map an exact host path into the container, under your control rather than Docker's.

```
docker run -d --name web -v /home/user/site:/usr/share/nginx/html:ro -p 8080:80 nginx
```

8.4.1 The Modern --mount Syntax

--mount is more explicit than **-v**, states the mount type, and rejects invalid combinations that **-v** would silently misread; it is recommended in scripts.

```
docker run -d --name web \
  --mount type=bind,source=/home/user/site,target=/usr/share/nginx/html,readonly \
  -p 8080:80 nginx
docker run -d --name db \
  --mount type=volume,source=mydata,target=/var/lib/postgresql/data postgres:16
```

8.4.2 Bind Mounts for Local Development

Mount source code so host edits appear instantly in the container, with no rebuild:

```
docker run -it --rm -v $(pwd):/app -w /app -p 3000:3000 node:20 npm run dev
```

With hot-reload this gives a fast dev loop inside a production-like container.

8.5 tmpfs Mounts (Linux Only)

`docker run -d --tmpfs /app/cache:rw,size=64m myapp`. Data lives only in RAM and vanishes when the container stops; use it for temporary sensitive data such as decrypted secrets.

8.6 Volume Drivers and Cloud Storage

Volume drivers allow network or cloud-backed volumes (NFS, Amazon EBS/EFS, Azure Files, distributed storage). This matters in Swarm and Kubernetes, where a container may land on any node and local volumes would not follow it.

```
docker volume create --driver local --opt type=nfs \
  --opt o=addr=192.168.1.100,rw --opt device=:/path/to/share nfs_volume
```

8.7 Backing Up and Restoring a Volume

Use a temporary helper container to tar the volume:

```
docker run --rm -v mydata:/data -v $(pwd):/backup \
  busybox tar czf /backup/mydata_backup.tar.gz -C /data .
docker volume create mydata_restored
docker run --rm -v mydata_restored:/data -v $(pwd):/backup \
  busybox tar xzf /backup/mydata_backup.tar.gz -C /data
```

8.8 Choosing Between Volumes and Bind Mounts

Question	Choose
Data must survive container removal?	Volume
Edit files live from the host in development?	Bind mount
Want Docker to manage the location?	Volume
Share a specific existing host directory (e.g., config)?	Bind mount
Running across multiple hosts?	Volume with a network-backed driver
Sensitive data that must never touch disk?	tmpfs

Chapter 9: Docker Compose — Multi-Container Applications

9.1 The Problem Compose Solves

A typical app needs a web/API server, a database, a cache such as Redis, and maybe a reverse proxy, all networked with the right startup order, volumes, and environment. Doing this by hand with many `docker network create`, `docker volume create`, and `docker run` commands is tedious, error-prone, and hard to share. Compose describes the whole stack declaratively in one YAML file and starts or stops it with one command.

9.2 Installing Compose

Modern Docker includes Compose as a plugin invoked as `docker compose` (space), the successor to the hyphenated Python tool `docker-compose`, now in maintenance mode. Check with `docker compose version`.

9.3 Anatomy of a docker-compose.yml File

```
services:          # each service becomes one container (or more, if scaled)
  web:
    build: ./web    # build from a local Dockerfile
    ports: ["8080:80"]
    environment:
      - NODE_ENV=production
      - DB_HOST=db
    depends_on: [db]
    networks: [appnet]
    restart: unless-stopped
  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=appdb
    volumes:
      - dbdata:/var/lib/postgresql/data
    networks: [appnet]
  cache:
    image: redis:7-alpine
    networks: [appnet]
volumes:           # top-level named volumes
  dbdata:
networks:          # top-level networks
  appnet:
    driver: bridge
```

(The old `version:` key is optional in modern Compose.)

9.3.1 Key Sections Explained

Key	Purpose
services	One entry per container/service
build / image	Build from a local Dockerfile context, or use a pre-built image
ports / environment / env_file	Same as <code>-p</code> , <code>-e</code> , <code>--env-file</code>
volumes	Service-level mounts; top-level <code>volumes:</code> declares named volumes
networks	Attach services to networks
depends_on	Startup order (not readiness; see 9.7)
restart	Same values as <code>docker run --restart</code>
command / entrypoint	Override the image's defaults

9.4 Core Compose Commands

```
docker compose up [-d] [--build]    # create/start (detached / force rebuild)
docker compose down                  # remove containers and networks, keep named volumes
docker compose down --volumes        # also remove volumes: DELETES DATA
docker compose ps | logs | logs -f SERVICE
docker compose exec SERVICE bash
docker compose build                  # build without starting
docker compose stop | start | restart SERVICE
docker compose config                 # validate and print resolved config
```

9.5 Scaling Services

`docker compose up -d --scale web=3` starts three replicas. A fixed host port (`8080:80`) makes scaling fail with a port conflict, so scaled services omit fixed host ports and sit behind a reverse proxy or use ephemeral ports.

9.6 Multiple Compose Files and Overrides

`docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`

- `docker-compose.yml` — base configuration.
- `docker-compose.override.yml` — merged automatically in development (bind-mount source, debug ports).
- `docker-compose.prod.yml` — production overrides (resource limits, no bind mounts, restart policy).

```
# docker-compose.override.yml
services:
  web:
    volumes: ["/web:/app"]
    command: npm run dev
```

9.7 depends_on and Startup Ordering

```
services:
  web:
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5
```

Plain `depends_on` guarantees only start order, not readiness; a database container can be "started" long before the engine is ready. `condition: service_healthy` (with a healthcheck) waits for real readiness. Applications should also retry with backoff, which works in any orchestrator.

9.8 Environment Variable Substitution

```
image: "myapp:${TAG:-latest}"      # defaults to latest if TAG unset
ports: ["${WEB_PORT}:80"]
# .env alongside the Compose file: TAG=1.4.2  WEB_PORT=8080
```

9.9 Worked Example: A Full-Stack App

```
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    depends_on: [api]
  api:
    build: ./api
    ports: ["5000:5000"]
    environment:
      - DATABASE_URL=postgres://postgres:secret@db:5432/appdb
      - REDIS_URL=redis://cache:6379
    depends_on:
      db: {condition: service_healthy}
      cache: {condition: service_started}
  db:
    image: postgres:16
    environment: [POSTGRES_PASSWORD=secret, POSTGRES_DB=appdb]
    volumes: ["dbdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 5
  cache:
    image: redis:7-alpine
volumes:
  dbdata:
```

All four services are networked with name-based discovery (**db**, **cache**, **api**) and start with **docker compose up -d**.

9.10 Compose vs. Swarm vs. Kubernetes

Compose targets a single host: local development, testing, small single-server production, and CI. It does not provide multi-host clustering, cross-machine failover, or fleet rolling updates. For that use Docker Swarm (which can consume a Compose file via **docker stack deploy**) or Kubernetes (Chapters 14 and 15).

Chapter 9b: Advanced Docker Compose Features

9b.1 Profiles — Selectively Enabling Services

Profiles let one file hold services that start only in certain contexts (debug tools, migration jobs). A service with no **profiles**: always starts.

```

services:
  web:    {image: myapp}
  adminer: {image: adminer, ports: ["8081:8080"], profiles: ["debug"]}
  migrate: {image: myapp-migrator, profiles: ["setup"]}
# docker compose up -d          -> web only
# docker compose --profile debug up -d    -> web + adminer
# docker compose --profile setup run migrate

```

9b.2 YAML Anchors for Reuse

```

x-common-env: &common-env
  LOG_LEVEL: info
  TZ: UTC
services:
  api:
    image: myapi
    environment: {<<: *common-env, SERVICE_NAME: api}
  worker:
    image: myworker
    environment: {<<: *common-env, SERVICE_NAME: worker}

```

9b.3 Resource Limits in Compose

```

services:
  api:
    image: myapi
    deploy:
      resources:
        limits:      {cpus: "1.0", memory: 512M}
        reservations: {memory: 256M}

```

Under plain `docker compose up` only `resources.limits` are enforced; the rest of `deploy:` (replicas, restart_policy, placement) matters mainly in Swarm mode.

9b.4 docker compose watch — Live Development Sync

```

services:
  api:
    build: ./api
    develop:
      watch:
        - {action: sync, path: ./api/src, target: /app/src}
        - {action: rebuild, path: ./api/package.json}

```

Run `docker compose watch`: changes in `./api/src` sync live into the container, while `package.json` changes trigger a rebuild.

9b.5 Dependency Conditions Beyond service_healthy

`condition: service_completed_successfully` makes a service wait for a one-shot job, such as a database migration, to finish successfully.

```

services:
  migrate: {image: myapp-migrator, depends_on: {db: {condition: service_healthy}}}
  api:     {image: myapi, depends_on: {migrate: {condition: service_completed_successfully}}}

```

9b.6 Named Compose Projects

Compose prefixes resources with a project name derived from the directory. Naming projects lets several copies of a stack run side by side, e.g., one per feature branch:

```
docker compose -p myapp-feature-x up -d and docker compose -p
myapp-feature-y up -d.
```

Chapter 10: Docker Registries and Image Distribution

10.1 What Is a Registry?

A registry stores and delivers images, organized into repositories (related images distinguished by tag) within namespaces (usually a user or organization). Reference anatomy: `docker.io / library / nginx : 1.25` = registry / namespace / repository / tag. `docker pull nginx` expands to `docker.io/library/nginx:latest`; `library` holds official, Docker-curated images.

10.2 Docker Hub

Docker Hub (hub.docker.com) is the default public registry, with:

- **Official Images** — curated, maintained images (nginx, postgres, node, python, redis) vetted for security and best practice.
- **Verified Publisher / Sponsored OSS** images from known organizations.
- **Public repositories** — free for anyone; **private repositories** — a few free, more with a paid plan.

10.2.1 Authenticating

```
docker login                # Docker Hub (a personal access token is recommended)
docker login myregistry.example.com # another registry
docker logout
```

10.3 Pushing and Pulling Images

```
docker pull nginx:1.25
docker pull myregistry.example.com/team/app:1.0
docker tag myapp:1.0 myusername/myapp:1.0      # tag with the target repository first
docker push myusername/myapp:1.0
docker tag myapp:1.0 myregistry.example.com/team/myapp:1.0
docker push myregistry.example.com/team/myapp:1.0
```

Transfers are efficient because only layers missing on the destination are sent (shared layers, Chapter 2).

10.4 Running a Private Registry

```
docker run -d -p 5000:5000 --name registry -v registry_data:/var/lib/registry registry:2
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
docker pull localhost:5000/myapp:1.0
```

Production registries need TLS and authentication and are often managed services:

Provider	Service
AWS	Elastic Container Registry (ECR)
Google Cloud	Artifact Registry / Container Registry
Microsoft Azure	Azure Container Registry (ACR)
GitHub / GitLab	GitHub Container Registry (GHCR) / GitLab Container Registry
Self-hosted	Harbor (adds vulnerability scanning, RBAC, replication)

10.5 Image Naming Conventions for Registries

[REGISTRY_HOST[:PORT]/]NAMESPACE/REPOSITORY[:TAG], for example `nginx:1.25` (Docker Hub official), `myusername/myapp:1.0` (Docker Hub personal), `ghcr.io/myorg/myapp:1.0` (GHCR), `123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0` (AWS ECR).

10.6 Image Digests and Immutability

Tags are mutable: `myapp:latest` can change over time. For reproducible deployments reference the immutable digest, `docker pull myapp@sha256:3f8a4d9e1b2c...`. CI/CD commonly resolves a tag to its digest at build time and deploys by digest, so exactly the tested artifact reaches production.

10.7 Vulnerability Scanning

Docker Hub, ECR, GHCR, and Harbor scan images against CVE databases. Failing the CI build on critical findings is a standard DevSecOps practice (Chapter 17). Docker's own tool: `docker scout cves myapp:1.0`.

10.8 Rate Limits and Caching

Docker Hub rate-limits pulls for anonymous and free users, so many CI systems and clusters run a pull-through cache/mirror in front of it to avoid redundant pulls and rate-limit failures.

Chapter 11: The Docker Engine, Deep Dive

11.1 Recap and Expansion

Chapter 2 introduced the client-server model and the containerd/runc chain. This chapter goes deeper into how the Engine isolates and runs containers and how it is configured and monitored.

11.2 The Full Component Stack

docker CLI	-> talks to the REST API
dockerd	-> images, build (BuildKit), networks, volumes; delegates execution
containerd	-> image pull/push, snapshots, container lifecycle; gRPC API (also used by Kubernetes' CRI)
containerd-shim-runc-v2 (one per container)	-> decouples the container's lifetime from containerd, reports exit status, keeps STDIO open
runc	-> OCI-compliant: clone() with namespace flags, sets up cgroups, pivots the root filesystem, execs the process
container process	-> PID 1 in its own PID namespace

11.3 BuildKit

Modern builds (`docker build`, `docker buildx build`) use **BuildKit**, a ground-up redesign (default since Docker 23.0):

- **Parallel execution** of independent stages in multi-stage builds.
- **Better caching**, including cache export/import to a registry so CI runners can share caches.
- **Build secrets** (`--secret`) mount a secret into one build step without persisting it in a layer, unlike `ARG/ENV`.
- **buildx** provides multi-platform builds (e.g., linux/amd64 and linux/arm64) via QEMU or native builders.

```
docker buildx build --platform linux/amd64,linux/arm64 -t myapp:1.0 --push .
docker build --secret id=npmc,src=$HOME/.npmrc -t myapp .
# Dockerfile: RUN --mount=type=secret,id=npmc cp /run/secrets/npmrc ~/.npmrc && npm install
```

11.4 Storage Drivers

A storage driver implements the layered copy-on-write filesystem. The default and recommended Linux driver is **overlay2** (kernel OverlayFS); aufs, devicemapper, btrfs, and zfs exist for legacy or specialized cases. Check with `docker info | grep "Storage Driver"`.

11.5 The Container Runtime Spec (OCI)

- **Image Specification** — the standard image format (manifest, layers, config), so a Docker-built image runs on podman, containerd, or a Kubernetes node unchanged.
- **Runtime Specification** — the contract for running a container (an on-disk bundle plus `config.json` for namespaces, mounts, process). **runc** implements it, as do drop-in alternatives such as crun and the sandboxed gVisor and Kata runtimes.

11.6 Daemon Configuration

The daemon reads `/etc/docker/daemon.json` on Linux (Docker Desktop settings on Mac/Windows):

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "default-address-pools": [ { "base": "172.30.0.0/16", "size": 24 } ],
  "insecure-registries": ["myregistry.local:5000"],
  "registry-mirrors": ["https://mirror.gcr.io"]
}
```

Restart afterwards with `sudo systemctl restart docker`.

11.7 Logging Drivers

The default `json-file` driver feeds `docker logs`. Production usually ships logs elsewhere:

Driver	Destination
json-file (default)	Local JSON files on the host
journald / syslog	systemd journal / a syslog server
fluentd	Fluentd, often forwarding to Elasticsearch/OpenSearch
awslogs / gelf	AWS CloudWatch Logs / Graylog or Logstash endpoint

Example: `docker run -d --log-driver=journald myapp`. **Without `max-size`/`max-file` limits, json-file logs grow unbounded and can fill the host disk**, a painful and common production incident. Always set rotation.

11.8 Resource Isolation and Limits Revisited

```
docker run -d --memory=512m --memory-swap=1g --cpus="1.5" \
  --cpu-shares=512 --pids-limit=100 myapp
```

- `--memory` — hard cap; the kernel's OOM killer terminates processes that exceed it.
- `--cpus` — maximum CPU (1.5 cores) even on a multi-core host.
- `--cpu-shares` — relative weight applied only under contention, not a hard cap.
- `--pids-limit` — caps processes/threads, guarding against fork bombs.

11.9 Security Features of the Engine

- **Namespaces and cgroups** — the base isolation primitives.
- **Linux capabilities** — Docker drops many root capabilities by default even for root containers; tune with `--cap-add`/`--cap-drop`.
- **Seccomp** — restricts system calls; a sensible default profile is applied.
- **AppArmor / SELinux** — mandatory access control to further confine processes.
- **Rootless mode** — `dockerd` itself runs as a non-root user, limiting the blast radius.
- **User namespaces** — map container root (UID 0) to an unprivileged host UID.

Chapter 11b: Beyond Docker — The Broader Container Ecosystem

Docker popularized containers, but the OCI standards it helped create now support several alternative and complementary tools.

11b.1 Podman

A daemonless engine developed mainly by Red Hat, largely CLI-compatible with Docker (`podman run`, `build`, `ps`). Differences: **no daemon** (each command manages containers directly, so no long-running privileged process); **rootless by default**, with no docker-group equivalent needed; **native pods** mirroring Kubernetes and `podman generate kube` to emit Kubernetes YAML; and no built-in Compose (use `podman-compose` or `podman compose`).

11b.2 containerd and nerdctl

containerd is Docker's own runtime and also used directly by Kubernetes. **nerdctl** is a Docker-compatible CLI for containerd, handy on Kubernetes nodes that only have containerd, with extras such as lazy-pulling and encrypted images.

11b.3 Buildah and Skopeo

Buildah builds OCI images without a full runtime, ideal for CI that only builds and pushes. **Skopeo** inspects, copies, and signs images across registries without pulling them or running a daemon.

11b.4 LXC/LXD

LXC predates Docker and offers "system containers" closer to lightweight VMs (a full init system and many processes) than Docker's one-process application containers. LXD manages LXC.

11b.5 gVisor and Kata Containers

Both implement the OCI runtime spec as drop-in alternatives to runc and address the shared-kernel weakness. **gVisor** (Google) intercepts system calls in a user-space sandbox at some performance cost. **Kata Containers** runs each container in its own fast micro-VM, giving VM-grade isolation at higher resource cost. Both suit multi-tenant, untrusted workloads such as public code-execution platforms and CI runners.

11b.6 Where Docker Fits Today

Docker remains the dominant on-ramp for learning containers and local development (Desktop, Hub, Compose, huge community), and the concepts (images, layers, namespaces, cgroups, OCI) carry over to every tool above.

Chapter 12: Docker on Windows and macOS

12.1 Why This Needs Its Own Chapter

Containers need a Linux kernel, and neither Windows nor macOS has one natively, so both add a virtualization layer. That layer explains many platform quirks.

12.2 Docker Desktop on macOS

Desktop runs a small purpose-built Linux VM (Apple's Virtualization.framework, replacing HyperKit). `dockerd` and all containers live in the VM; the Mac's `docker` CLI reaches it through a forwarded socket.

12.2.1 Practical Implications

- **File sharing:** bind mounts cross the VM boundary and were historically slow, especially for many small files (`node_modules`). VirtioFS has narrowed the gap.
- **Networking:** published ports on localhost are forwarded into the VM, so `-p 8080:80` just works.
- **Resources:** VM CPU, memory, and disk are set in Settings > Resources; containers cannot exceed them.
- **Apple Silicon (arm64) vs Intel (amd64):** amd64-only images run under QEMU emulation (slower); many images now ship multi-platform manifests with native arm64 builds.

12.3 Docker Desktop on Windows

12.3.1 WSL2 Backend (Recommended)

WSL2 runs a real lightweight Linux kernel, integrated far more tightly than a classic VM; Desktop installs its daemon in a dedicated WSL2 distribution. Any installed WSL2 distro (Ubuntu, Debian) can be enabled for Docker integration for near-native performance. **Keep project files in the WSL2 filesystem** (`/home/user/project`), not on Windows via `/mnt/c/...`, since crossing ext4 and NTFS is slow.

12.3.2 Hyper-V Backend (Legacy)

A heavier traditional VM, largely superseded by WSL2 but still used where enterprise policy blocks WSL2.

12.3.3 Windows Containers

Desktop can also run native **Windows containers** (Windows Server Core or Nano Server base images, using Windows isolation) after switching to "Windows containers" mode; they serve legacy .NET Framework apps. Windows and Linux containers are not interchangeable. Everything else in these notes concerns Linux containers.

12.4 Common Cross-Platform Gotchas

Issue	Cause	Fix
Slow bind-mount I/O	Filesystem crosses the VM boundary	Keep files inside WSL2 (Windows); rely on VirtioFS (Mac); avoid bind-mounting huge dependency dirs
<code>--network host</code> behaves differently	Shares the VM's network namespace, not the physical host's	Prefer explicit <code>-p</code> publishing
Line-ending / permission mismatches	Windows CRLF; NTFS/WSL2 permission models differ	Set <code>core.autocrlf</code> ; enforce LF via <code>.gitattributes</code>
Case sensitivity surprises	macOS/Windows filesystems are case-insensitive; Linux is not	Use consistent casing in COPY/import paths
Containers capped unexpectedly	Fixed VM CPU/RAM allocation	Raise it in Desktop > Settings > Resources

12.5 When to Just Use Linux

CI runners and production servers run Docker Engine natively on Linux, with no VM layer. Development setups therefore aim to mirror production Linux as closely as possible to avoid surprises.

Chapter 13: Container Orchestration — Why and What

13.1 From One Container to a Fleet

Everything so far works on one machine, but production usually needs many, for:

- **Capacity** — one host has finite CPU/RAM; scale horizontally.
- **High availability** — replicas across machines (ideally zones) remove a single point of failure.
- **Zero-downtime rolling updates** — replace old containers gradually.
- **Efficient scheduling** — decide which machine runs each container.

An **orchestrator** does all of this with minimal manual intervention.

13.2 What an Orchestrator Actually Does

Responsibility	Description
Scheduling	Chooses the node for each container based on resources and constraints
Service discovery and load balancing	Stable routing between services as instances come and go
Self-healing	Detects failed containers/nodes and restarts or reschedules workloads
Scaling	Raises or lowers replicas manually or automatically with load

Responsibility	Description
Rolling updates and rollbacks	Incremental replacement, reverting if health checks fail
Configuration and secrets	Distributes config and credentials to the containers that need them
Storage orchestration	Attaches persistent storage regardless of the node

13.3 The Two Major Orchestrators Covered

1. **Docker Swarm** (Chapter 14) — built into Docker; simpler, integrated with the CLI and Compose format, suited to smaller clusters.
2. **Kubernetes / k8s** (Chapter 15) — from Google, a CNCF graduated project and the de facto standard, with a far larger feature set, ecosystem, and learning curve.

The choice usually comes down to team size and expertise, ecosystem needs (Kubernetes has vastly more tooling, operators, and managed offerings), and tolerance for operational complexity.

13.4 A Cluster's Building Blocks

- **Node** — one machine in the cluster.
- **Manager / control-plane node** — makes cluster-wide decisions: scheduling, state, API access (Swarm "manager"; Kubernetes "control plane").
- **Worker node** — runs application containers as directed.
- **Service** — an abstraction over replicated identical instances with stable networking.
- **Desired-state reconciliation** — you declare "5 replicas," and the orchestrator continuously corrects drift (a crashed container is replaced). This declarative model, rather than imperatively fixing problems, is the key shift from plain **docker run**/Compose.

Chapter 14: Docker Swarm

14.1 What Is Swarm?

Swarm is Docker's native clustering and orchestration, built into the Engine. A group of Docker hosts becomes one logical swarm, managed with the familiar **docker** CLI plus swarm subcommands.

14.2 Initializing a Swarm

```
docker swarm init --advertise-addr <MANAGER_IP>           # first machine = manager
docker swarm join --token <WORKER_TOKEN> <MANAGER_IP>:2377 # on each worker
docker swarm join-token manager                           # token for extra managers
docker node ls | inspect NODE                             # on a manager
docker node update --availability drain NODE              # stop scheduling (maintenance)
docker swarm leave [--force]                              # leave (force for a manager)
```

14.3 Manager Nodes and Raft Consensus

Managers keep cluster state (services, replica counts, placements) using the **Raft** consensus algorithm. Use an odd number of managers (3 or 5): Raft needs a majority (quorum), odd counts avoid ties, and the swarm tolerates losing $(N-1)/2$ managers. Workers do not take part in consensus and only run tasks; a manager also runs workloads unless drained.

14.4 Services — The Core Swarm Abstraction

A **service** is a declarative spec (image, replicas, config) that Swarm continuously reconciles.

```
docker service create --name web --replicas 3 --publish 8080:80 nginx:1.25
docker service ls          # all services
docker service ps web      # tasks and the node each runs on
docker service inspect web
docker service logs web    # aggregated across replicas
```

14.4.1 Scaling a Service

`docker service scale web=5` schedules extra replicas on nodes with capacity and replaces any that die.

14.4.2 Rolling Updates

```
docker service update --image nginx:1.26 --update-parallelism 1 --update-delay 10s web
docker service update --rollback web          # manual rollback
# stack file:
deploy:
  update_config: {parallelism: 1, delay: 10s, failure_action: rollback, monitor: 30s}
```

Replicas are replaced incrementally (here one at a time, 10 seconds apart), keeping the service available; failed health checks can trigger automatic rollback.

14.5 The Routing Mesh and Load Balancing

A published port is reachable on that port on **every** swarm node, even those with no replica; the ingress routing mesh forwards traffic to a node running a healthy replica. Service-to-service calls by name use an embedded DNS virtual IP that load-balances across healthy replicas.

14.6 Stacks — Deploying Compose Files to Swarm

A **stack** deploys a multi-service app from a Compose-compatible file in one command, building on Chapter 9.

```
# docker-stack.yml
services:
  web:
    image: myrepo/web:1.0
    ports: ["8080:80"]
    deploy:
      replicas: 3
      restart_policy: {condition: on-failure}
      resources: {limits: {cpus: "0.5", memory: 256M}}
    networks: [appnet]
  db:
    image: postgres:16
    environment: [POSTGRES_PASSWORD=secret]
    volumes: ["dbdata:/var/lib/postgresql/data"]
    deploy:
      replicas: 1
      placement: {constraints: [node.role == manager]}
    networks: [appnet]
volumes: {dbdata: {}}
networks: {appnet: {driver: overlay}}
```

```
docker stack deploy -c docker-stack.yml myapp
docker stack ls | services myapp | ps myapp | rm myapp
```

The **deploy** key is Swarm-specific (ignored by plain **docker compose up**), and the **overlay** network is what lets containers on different nodes talk as if on one network.

14.7 Secrets and Configs

Swarm provides encrypted-at-rest primitives, better than environment variables for sensitive data:

```
echo "supersecretpassword" | docker secret create db_password -
docker secret ls
# service: environment: [POSTGRES_PASSWORD_FILE=/run/secrets/db_password]
#           secrets: [db_password]      (top-level: db_password: {external: true})
```

Secrets are mounted as in-memory files at **/run/secrets/<name>** and delivered only to nodes running a task that references them.

14.8 Placement Constraints and Node Labels

```
docker node update --label-add zone=us-east node1
deploy:
  placement:
    constraints: [node.labels.zone == us-east, node.role == worker]
```

Constraints pin stateful services to nodes with storage or spread replicas across failure domains.

14.9 When Swarm Is a Good Fit

Swarm fits teams that know Docker and Compose, need multi-host orchestration without a steep curve, and do not need Kubernetes' custom resources, operator ecosystem, or fine-grained RBAC. Many small-to-medium deployments run on Swarm for years, but Kubernetes' dominance has shrunk Swarm's ecosystem and momentum, a consideration for new long-lived projects.

Chapter 15: Kubernetes Fundamentals

15.1 What Is Kubernetes and Why Learn It Here

Kubernetes (Greek for "helmsman"; k8s = "k", 8 letters, "s") is an open-source orchestrator designed at Google (from its internal Borg system), donated to the CNCF in 2015, and now the industry standard. Docker-built images are exactly what Kubernetes runs, and it is the default target for deploying containerized apps at scale. It is far richer, and harder to operate, than Swarm.

15.2 Kubernetes Architecture

15.2.1 Control Plane Components

- **kube-apiserver** — the front door: the REST API everything, including `kubectl`, talks to.
- **etcd** — distributed key-value store holding all cluster state (the source of truth).
- **kube-scheduler** — assigns unscheduled Pods to nodes using resources, constraints, and affinity/anti-affinity rules.
- **kube-controller-manager** — runs controllers that drive actual state toward desired state (e.g., the ReplicaSet controller keeps the right Pod count).
- **cloud-controller-manager** — integrates with AWS, GCP, or Azure for load balancers and volumes.

15.2.2 Node (Worker) Components

- **kubelet** — agent on each node ensuring the Pods' containers run and stay healthy.
- **kube-proxy** — maintains network rules implementing Service load balancing (15.5).
- **Container runtime** — accessed via the Container Runtime Interface (CRI); typically containerd or CRI-O. Since Kubernetes 1.24 the "dockershim" is gone, so Kubernetes no longer talks to the full Docker Engine, but Docker-built images stay fully compatible because they are OCI images.

```
Control Plane: kube-apiserver -- etcd -- scheduler -- controller-manager
                | API calls
+-----+-----+-----+
Worker Node   Worker Node   ... Worker Node
kubelet, kube-proxy, containerd, [Pods]
```

15.3 The Pod — Kubernetes' Smallest Deployable Unit

Kubernetes schedules **Pods**, not containers: one or more containers always placed on the same node, sharing one network namespace (one IP, talking over localhost) and optionally volumes. Most Pods hold one container; multi-container patterns include **sidecars** (e.g., a log shipper) and **init containers** that run to completion first (waiting for a dependency, running a migration).

```

apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels: {app: myapp}
spec:
  containers:
  - name: myapp
    image: myrepo/myapp:1.0
    ports: [{containerPort: 8080}]
    resources:
      requests: {cpu: "250m", memory: "128Mi"}
      limits: {cpu: "500m", memory: "256Mi"}

```

```

kubectl apply -f pod.yaml
kubectl get pods | describe pod myapp-pod | logs myapp-pod
kubectl exec -it myapp-pod -- bash
kubectl delete pod myapp-pod

```

Pods are ephemeral and self-heal only under a controller: a bare Pod that dies is gone. Deployments and StatefulSets manage Pods for you.

15.4 Deployments — Managing Replicated, Self-Healing Pods

A Deployment declares a Pod template and replica count; Kubernetes keeps that many healthy copies running and performs rolling updates when the template changes.

```

apiVersion: apps/v1
kind: Deployment
metadata: {name: myapp}
spec:
  replicas: 3
  selector: {matchLabels: {app: myapp}}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxUnavailable: 1, maxSurge: 1}
  template:
    metadata: {labels: {app: myapp}}
    spec:
      containers:
      - name: myapp
        image: myrepo/myapp:1.0
        ports: [{containerPort: 8080}]
        readinessProbe:
          httpGet: {path: /health, port: 8080}
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet: {path: /health, port: 8080}
          initialDelaySeconds: 15
          periodSeconds: 20

```

```

kubectl apply -f deployment.yaml
kubectl get deployments | replicasetsets # Deployment -> ReplicaSet -> Pods
kubectl scale deployment myapp --replicas=5
kubectl set image deployment/myapp myapp=myrepo/myapp:1.1 # rolling update
kubectl rollout status | history | undo deployment/myapp

```

15.4.1 Readiness vs. Liveness Probes

- **Readiness** — is the Pod ready for traffic? On failure it leaves the Service's load-balancing pool but is not killed (slow startup, temporary overload).
- **Liveness** — is the Pod healthy? On failure the kubelet kills and restarts it.

This parallels Compose's `service_healthy` and Docker's `HEALTHCHECK` and underpins Kubernetes self-healing.

15.5 Services — Stable Networking for Ephemeral Pods

Pods get new IPs whenever replaced, so a **Service** gives a stable virtual IP and DNS name that load-balances across Pods matching a label selector.

```
apiVersion: v1
kind: Service
metadata: {name: myapp-service}
spec:
  selector: {app: myapp}
  ports: [{port: 80, targetPort: 8080}]
  type: ClusterIP
```

15.5.1 Service Types

Type	Behavior
ClusterIP (default)	Internal-only virtual IP within the cluster
NodePort	Also exposes a static port on every node's IP, reachable externally
LoadBalancer	Provisions a cloud load balancer (e.g., AWS ELB) with a public IP
ExternalName	Maps the service to an external DNS name

`kubectl apply -f service.yaml, kubectl get services, kubectl expose deployment myapp --port=80 --target-port=8080 --type=LoadBalancer.`

15.6 Ingress — HTTP(S) Routing Into the Cluster

A LoadBalancer exposes one service; real apps need host- and path-based routing (api.example.com to the API, example.com/admin to the admin service) and central TLS termination. An **Ingress** resource, implemented by an Ingress Controller (NGINX, Traefik), does this.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations: {nginx.ingress.kubernetes.io/rewrite-target: /}
spec:
  rules:
    - host: myapp.example.com
      http:
        paths:
          - {path: /, pathType: Prefix,
            backend: {service: {name: myapp-service, port: {number: 80}}}}
  tls:
    - {hosts: ["myapp.example.com"], secretName: myapp-tls}
```

15.7 ConfigMaps and Secrets

Like Compose environment/env_file and Swarm secrets, these keep configuration out of images.

```
apiVersion: v1
kind: ConfigMap
metadata: {name: myapp-config}
data: {APP_ENV: production, LOG_LEVEL: info}
---
apiVersion: v1
kind: Secret
metadata: {name: myapp-secret}
type: Opaque
stringData: {DB_PASSWORD: supersecret}
# in a Pod spec:  envFrom: [{configMapRef: {name: myapp-config}},
#                      {secretRef: {name: myapp-secret}}]
```

15.8 Namespaces

Namespaces split one cluster into virtual clusters, commonly per environment (dev, staging, production) or team, each with resource quotas and RBAC boundaries.

```
kubectl create namespace staging
kubectl apply -f deployment.yaml -n staging
kubectl get pods -n staging
kubectl config set-context --current --namespace=staging  # change default
```

15.9 Persistent Storage

A **PersistentVolume (PV)** is a real piece of storage (admin-provisioned or dynamically created via a storage class); a **PersistentVolumeClaim (PVC)** is a Pod's request that Kubernetes binds to a matching PV.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: myapp-data}
spec:
  accessModes: ["ReadWriteOnce"]
  resources: {requests: {storage: 10Gi}}
  storageClassName: standard
# in the Pod spec:
#   volumes: [{name: data, persistentVolumeClaim: {claimName: myapp-data}}]
#   containers: [{name: myapp, volumeMounts: [{mountPath: /data, name: data}]}]
```

15.10 kubectl Quick Reference

```
kubectl get pods,deployments,services,nodes    # list resources
kubectl get pods -o wide                       # includes node placement
kubectl describe pod POD                      # status and recent events
kubectl logs POD [-c CONTAINER]
kubectl exec -it POD -- sh
kubectl apply -f FILE.yaml | delete -f FILE.yaml
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl top pods                               # needs metrics-server
kubectl config get-contexts | use-context NAME
```

15.11 Helm — Packaging Kubernetes Applications

Helm is the de facto Kubernetes package manager: apps are versioned, parameterized **charts** installed, upgraded, and rolled back as one unit, like Compose for Kubernetes but with templating.

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-postgres bitnami/postgresql
helm upgrade my-postgres bitnami/postgresql --set auth.password=newpass
helm rollback my-postgres 1
helm uninstall my-postgres
```

15.12 Local Kubernetes for Development and Learning

- **Docker Desktop Kubernetes** — one click, single node in Desktop's VM.
- **Minikube** — dedicated single-node cluster for local development and tutorials.
- **kind** (Kubernetes IN Docker) — nodes run as containers; fast, disposable, popular in CI.
- **k3s / k3d** — lightweight certified distribution for edge and constrained environments; k3d is the Docker wrapper.

15.13 Docker Swarm vs. Kubernetes — Summary Comparison

Aspect	Docker Swarm	Kubernetes
Learning curve	Gentle; reuses Docker/Compose knowledge	Steep; large vocabulary
Setup	Simple (<code>docker swarm init</code>), built in	Significant (eased by EKS, GKE, AKS); separate install
Ecosystem	Smaller, less active	Vast (Helm, operators, service meshes, CNCF)
Scalability	Small/medium clusters	Proven at very large scale
Adoption trend	Declining	Dominant, growing
Best fit	Small teams, simple deployments, fast onboarding	Large organizations, complex systems, cloud-native

Chapter 16: CI/CD and Deploying Containerized Applications

16.1 The Full Pipeline, End to End

A change travels from commit to production with the container image as the artifact throughout:

```
Developer --git push--> CI server: 1 checkout, 2 unit tests, 3 docker build
--4 docker push--> Registry
--5 pull new image--> Orchestrator --6 rolling update--> Production
```

16.2 Continuous Integration: Building and Testing the Image

```
# .github/workflows/ci.yml
name: CI
on: {push: {branches: [main]}}
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests      # multi-stage Dockerfile with a "test" stage
        run: docker build --target test -t myapp:test .
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}
      - uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ghcr.io/myorg/myapp:${GITHUB_SHA}
            ghcr.io/myorg/myapp:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Key patterns:

- **Tag by commit SHA**, not just **latest**: unique, immutable, traceable to source (Chapter 10).
- **Registry/CI build cache** (**cache-from/cache-to**) reuses layers across runs via BuildKit (Chapter 11).
- A **test stage** in the multi-stage Dockerfile (Chapter 6) lets the production Dockerfile also run tests in CI.

16.3 Continuous Deployment: Rolling Out the New Image

- **Swarm**: `docker service update --image ghcr.io/myorg/myapp:$GIT_SHA myapp_web`
- **Kubernetes**: `kubectl set image deployment/myapp myapp=ghcr.io/myorg/myapp:$GIT_SHA` then `kubectl rollout status deployment/myapp`. In mature setups use **GitOps**: ArgoCD or Flux watches a Git repo of manifests (including the image tag) and applies changes, so deployment means committing a new tag and everything is auditable in Git history.
- **Single server**: `ssh deploy@server "docker pull ghcr.io/myorg/myapp:$GIT_SHA && docker compose -f /opt/myapp/docker-compose.yml up -d"`

16.4 Deployment Strategies

Strategy	Description	Trade-off
Recreate	Stop all old, then start new	Simple, but downtime
Rolling update	Replace instances incrementally	No downtime; versions briefly coexist

Strategy	Description	Trade-off
Blue-Green	Run new version fully beside old, switch all traffic at once	Instant rollback; double resources temporarily
Canary	Send a small traffic share to the new version, then increase	Limits blast radius; needs finer traffic routing

Blue-green and canary usually need Kubernetes with a service mesh (Istio, Linkerd) or weighted cloud load balancing, beyond plain Compose or basic Swarm.

16.5 Health Checks Tie the Whole Pipeline Together

HEALTHCHECK (Ch. 6), Compose **service_healthy** (Ch. 9), the Swarm routing mesh (Ch. 14), and Kubernetes probes (Ch. 15) are one idea at different layers: tooling must know when a container is *ready*, not just started. Safe automated rollouts and rollbacks depend on accurate health signals.

16.6 Observability in Production

- **Logs** — via a log driver (Ch. 11) or sidecar/DaemonSet shipper into ELK/EFK or a managed service (Datadog, CloudWatch).
- **Metrics** — scraped by Prometheus, shown in Grafana; **docker stats** and **kubectl top** are the ad hoc equivalents.
- **Tracing** — OpenTelemetry or Jaeger follows one request across microservices to find latency.
- **Alerting** — rules such as Prometheus Alertmanager page engineers before users complain.

Chapter 16b: Capstone Project — Containerizing and Deploying a Three-Tier Application

A complete example applying most earlier concepts: a React frontend, a Node.js/Express API, and a PostgreSQL database, from local development to an orchestrated production deployment.

16b.1 Project Layout

```
myapp/
  frontend/ (Dockerfile, package.json, src/)
  api/      (Dockerfile, package.json, src/)
  docker-compose.yml  docker-compose.override.yml  docker-stack.yml
  .dockerignore      .env.example
```

16b.2 The API Dockerfile

Multi-stage build, cache-friendly ordering, non-root user, and an ENTRYPOINT/CMD split:

```

FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS production
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
ENV NODE_ENV=production
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package*.json ./
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD node -e "require('http').get('http://localhost:5000/health', \
    r => process.exit(r.statusCode===200?0:1))"
ENTRYPOINT ["node"]
CMD ["dist/server.js"]

```

16b.3 The Frontend Dockerfile

```

FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.25-alpine AS production
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK --interval=30s CMD wget -qO- http://localhost/ || exit 1
CMD ["nginx", "-g", "daemon off;"]

```

16b.4 Local Development with Compose

```
# docker-compose.yml (base)
services:
  frontend: {build: ./frontend, ports: ["3000:80"], depends_on: [api], networks: [appnet]}
  api:
    build: ./api
    ports: ["5000:5000"]
    environment: ["DATABASE_URL=postgres://postgres:${DB_PASSWORD}@db:5432/appdb"]
    depends_on: {db: {condition: service_healthy}}
    networks: [appnet]
  db:
    image: postgres:16
    environment: ["POSTGRES_PASSWORD=${DB_PASSWORD}", "POSTGRES_DB=appdb"]
    volumes: ["dbdata:/var/lib/postgresql/data"]
    healthcheck: {test: ["CMD-SHELL", "pg_isready -U postgres"], interval: 5s, retries: 5}
    networks: [appnet]
volumes: {dbdata: {}}
networks: {appnet: {driver: bridge}}

# docker-compose.override.yml (auto-loaded locally): bind-mount source, run dev servers
services:
  frontend: {build: {target: build}, volumes: ["/frontend/src:/app/src"],
    command: npm run dev}
  api: {build: {target: build}, volumes: ["/api/src:/app/src"], command: npm run dev,
    environment: ["NODE_ENV=development"]}

cp .env.example .env          # fill in DB_PASSWORD, etc.
docker compose up --build     # live-reloading dev loop
```

16b.5 CI: Build, Test, Push

A GitHub Actions matrix builds both services, logs in to GHCR, and pushes with `docker/build-push-action`, tagging `ghcr.io/myorg/myapp-${{ matrix.service }}:${{ github.sha }}` with `cache-from/cache-to: type=gha` (same pattern as 16.2, using `strategy: matrix: {service: [frontend, api]}` and `context: ./${{ matrix.service }}`).

16b.6 Production Deployment: Docker Swarm Stack

```
# docker-stack.yml
services:
  frontend:
    image: ghcr.io/myorg/myapp-frontend:${IMAGE_TAG}
    ports: ["80:80"]
    deploy:
      replicas: 2
      update_config: {parallelism: 1, delay: 10s, failure_action: rollback}
      networks: [appnet]
  api:
    image: ghcr.io/myorg/myapp-api:${IMAGE_TAG}
    deploy:
      replicas: 3
      update_config: {parallelism: 1, delay: 10s, failure_action: rollback}
      resources: {limits: {cpus: "0.5", memory: 256M}}
      secrets: [db_password]
      environment: ["DATABASE_URL=postgres://postgres@db:5432/appdb"]
      networks: [appnet]
  db:
    image: postgres:16
    deploy: {replicas: 1, placement: {constraints: ["node.role == manager"]}}
    secrets: [db_password]
    volumes: ["dbdata:/var/lib/postgresql/data"]
    networks: [appnet]
secrets: {db_password: {external: true}}
volumes: {dbdata: {}}
networks: {appnet: {driver: overlay}}

docker secret create db_password db_password.txt
IMAGE_TAG=$(git rev-parse --short HEAD) docker stack deploy -c docker-stack.yml myapp
docker stack services myapp
```

16b.7 What This Project Demonstrates

Multi-stage builds for lean, secure images (Ch. 6); non-root users and healthchecks (Ch. 6, 17); user-defined networking with name discovery (Ch. 7); a named volume for the database (Ch. 8); dev/production split through Compose layering (Ch. 9); immutable SHA-tagged images from CI (Ch. 10, 16); and zero-downtime rolling deploys with automatic rollback, secrets, and placement constraints on Swarm (Ch. 14). The same app could deploy to Kubernetes by translating the stack file into Deployment, Service, Ingress, and Secret manifests (Ch. 15): once properly containerized, the orchestrator is a deployment-time decision.

Chapter 17: Best Practices, Security, and Common Pitfalls

17.1 Dockerfile Best Practices Checklist

- **Pin minimal base images** (`python:3.12.4-slim`, not `latest`); prefer slim/alpine/distroless.
- **Order instructions least to most volatile** (dependencies before source) for cache hits.
- **Use multi-stage builds** and a `.dockerignore` to keep tools, secrets, and dev files out.
- **Prefer exec-form CMD/ENTRYPOINT** so the app receives signals and shuts down gracefully.

- **Run as non-root:** `RUN groupadd -r appuser && useradd -r -g appuser appuser` then `USER appuser`.
- **Set resource limits** (`--memory`, `--cpus`, or Compose/Kubernetes equivalents) so one container cannot starve others.
- **Add a HEALTHCHECK** so orchestrators and `docker ps` know real health.
- **Never bake secrets into layers:** not via `ENV`, and not via a `COPY`'d credentials file deleted later (deletion does not remove it from earlier layers). Use `--secret`, runtime variables, or a secrets manager.

17.2 Image Security

17.2.1 Minimize the Attack Surface

Every installed package is a potential vulnerability. Install only what is needed and clean caches in the same layer:

```
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

17.2.2 Scan Images for Known Vulnerabilities

Run Docker Scout, Trivy, Snyk, or a registry scanner in CI and fail the build on critical/high findings: `trivy image myapp:1.0`.

17.2.3 Don't Run Containers as Root Unnecessarily

Root in a container is not automatically catastrophic, but a container-escape bug plus a root process is strictly worse than the same bug plus an unprivileged one. Combine a non-root `USER` with user-namespace remapping (Ch. 11).

17.2.4 Avoid --privileged and Unnecessary Capabilities

`docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE myapp`. `--privileged` grants nearly all capabilities and device access; reserve it for infrastructure tooling, never ordinary apps.

17.2.5 Read-Only Root Filesystems

`docker run --read-only --tmpfs /tmp myapp` prevents an attacker with code execution from persisting files or tampering with binaries.

17.2.6 Keep the Engine and Host Patched

Flaws in runc, containerd, the daemon, or the host kernel can allow container escape, so patch them like application dependencies.

17.3 Networking and Secrets Hygiene

- Publish only necessary ports and bind to minimal interfaces.
- Segment unrelated apps with user-defined networks, not the default bridge (Ch. 7).
- Use a real secrets manager (Swarm secrets, Kubernetes Secrets backed by Vault or a cloud service) instead of plain environment variables or `.env` in production.
- Rotate credentials and never log secrets or put them in error messages.

17.4 Operational Best Practices

- Set `json-file` log rotation or ship logs off-host (Ch. 11).
- Define requests/limits for every production service, to stop noisy neighbors and inform the scheduler.
- Deploy by digest, resolving tags at deploy time (Ch. 10).
- Automate: production flows through version-controlled manifests and CI/CD, never ad hoc commands on a host.
- Handle SIGTERM gracefully within the grace period (default 10s in Docker) before SIGKILL.
- Prune long-lived hosts regularly, but never blindly use `--volumes` in production.

17.5 Common Beginner Mistakes, Revisited

Mistake	Why it's a problem	Fix
<code>latest</code> tag in production	Non-reproducible, silent changes	Pin versions or digests
State in the writable layer	Data lost on removal	Use volumes (Ch. 8)
Everything runs as root	Weakens the security boundary	Add a non-root USER
One giant single-stage Dockerfile	Bloated image, slow builds	Multi-stage builds
Hardcoded configuration	Same image can't move across environments	Environment variables (Ch. 5)
No <code>.dockerignore</code>	Slow builds, secret leakage	Add a thorough <code>.dockerignore</code>
No health checks	Orchestrators can't tell if it works	HEALTHCHECK / probes
SSHing in to deploy by hand	Error-prone, unrepeatable, unauditabile	Automate via CI/CD (Ch. 16)

Chapter 17b: Troubleshooting Docker — A Practical Field Guide

Common failure modes organized by symptom.

17b.1 "Cannot connect to the Docker daemon"

Message: `Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?`

```
sudo systemctl status docker      # running?
sudo systemctl start docker
ls -la /var/run/docker.sock       # socket and permissions
groups $USER                      # in the docker group?
```

Causes: daemon crashed or never started; user not in the `docker` group (Ch. 3); Docker Desktop not running; daemon listening on a different socket or port than the client expects.

17b.2 Container Exits Immediately After docker run

Check `docker ps -a` (Exited (0) or (1)), `docker logs CONTAINER`, and `docker inspect CONTAINER --format='{{.State.ExitCode}}'`. Causes: PID 1 finished (expected for `sleep 5`, wrong for services); missing `-it` for an interactive shell; startup crash from missing environment variables (Ch. 5), an unready dependency (Ch. 9), or a file never COPY'd.

17b.3 "Port is already allocated"

Error: `Bind for 0.0.0.0:8080 failed: port is already allocated`. Find the owner with `docker ps --filter "publish=8080"` or `sudo lsof -i :8080`, then stop it or choose another host port (`-p 8081:80`).

17b.4 Container Can't Reach Another Container by Name

1. Are both on the same network? (`docker network inspect NETWORK`)
2. Is it the default bridge? Name resolution doesn't work there; use a user-defined network.
3. Is the target listening on `0.0.0.0`, not `127.0.0.1`?
4. Is it healthy yet? Add a healthcheck with `service_healthy` (Ch. 9) or client retry logic.

17b.5 "No space left on device"

```
docker system df [-v]              # what's consuming space
docker system prune -a --volumes   # aggressive: also deletes unused VOLUMES
```

Look for root causes: unbounded logs (Ch. 11), CI runners accumulating cache and images, or too little disk (on Mac/Windows, the Desktop VM's virtual disk).

17b.6 Build Is Slow or Cache Never Hits

Run `docker build --progress=plain -t myapp .` to see which layers rebuild. Causes: `COPY . .` before dependency install; no `.dockerignore` (editor files or `.git` change the context hash); needless `--no-cache`; a mutable upstream tag like `:latest`.

17b.7 "Permission denied" Inside a Container

Usually a UID/GID mismatch on bind mounts, especially with a non-root image user. Match IDs: `docker run --user $(id -u):$(id -g) -v $(pwd):/app myimage`; or create the image user with a matching UID, or fix host ownership.

17b.8 Image Builds Locally but Fails in CI

Causes: different build context (`.dockerignore`, uncommitted `.env`); architecture mismatch (arm64 locally, amd64 in CI; use `docker buildx build --platform`); no credentials or network for a private base image; non-deterministic steps such as pulling `:latest`.

17b.9 Container Runs Out of Memory (OOMKilled)

`docker inspect CONTAINER --format='{{.State.OOMKilled}}'` and `docker stats CONTAINER`. Either the app leaks memory (profile it) or the limit (Ch. 11, or Kubernetes limits, Ch. 15) is too low for real peak usage; measure under realistic load and leave headroom.

17b.10 Swarm/Kubernetes Deployment Stuck ("Pending" or Not Updating)

`docker service ps SERVICE --no-trunc` (Swarm) or `kubectl describe pod POD` (see Events at the bottom). Causes: insufficient cluster resources; image pull failure (wrong tag, missing `imagePullSecrets`); failing readiness/liveness probes blocking the rollout; placement constraints no node satisfies.

Chapter 18: Conclusion and Next Steps

18.1 What You've Learned

Why containers exist and how namespaces and cgroups give VM-like isolation cheaply (Ch. 1); Docker's architecture (Ch. 2, 11); the CLI (Ch. 4); twelve-factor configuration (Ch. 5); Dockerfiles, caching, multi-stage builds, CMD/ENTRYPOINT (Ch. 6); networking (Ch. 7); persistent storage (Ch. 8); Compose (Ch. 9); registries (Ch. 10); Windows/macOS nuances (Ch. 12); Swarm and Kubernetes with shared orchestration concepts (Ch. 13-15); CI/CD, deployment strategies, and observability (Ch. 16); and security hardening (Ch. 17).

18.2 Suggested Learning Path Going Forward

1. Containerize a personal project end to end (Dockerfile plus a Compose file for the database and cache) until the CLI is muscle memory.
2. Run a local Kubernetes cluster (kind or minikube) and redeploy it with Deployments, Services, and Ingress.
3. Build a real CI/CD pipeline (GitHub Actions or GitLab CI) that builds, tests, pushes, and deploys to a small server, Swarm, or managed Kubernetes (EKS/GKE/AKS).
4. Study a managed Kubernetes offering in depth; most production Kubernetes runs on a cloud-managed control plane.
5. Explore Helm, service meshes (Istio, Linkerd), GitOps (ArgoCD, Flux), and Prometheus/Grafana.
6. Deepen security knowledge: Trivy, runtime monitoring with Falco, and the CIS Docker/Kubernetes Benchmarks.

18.3 Closing Thought

Containers and DevOps exist to make it fast, safe, and repeatable to get software from a developer's editor to real users. Layered images, isolated networks, declarative orchestration, and automated pipelines all serve that goal. Tools will evolve; the underlying problem and principles remain the foundation worth understanding deeply.

Appendix A: Complete Command Cheat Sheet

```

# Container lifecycle
docker run [OPTIONS] IMAGE [CMD]    docker create IMAGE
docker start|stop|restart|pause|unpause|kill|rm [-f] CONTAINER
# Inspection and debugging
docker ps [-a] [-q]                docker inspect CONTAINER|IMAGE
docker logs [-f] [--tail N] CONTAINER    docker exec -it CONTAINER sh|bash
docker top CONTAINER                docker stats [CONTAINER]
docker cp SRC DEST                  docker diff CONTAINER    # files changed vs. the image
# Images
docker images | pull | push IMAGE[:TAG]    docker build -t NAME:TAG PATH
docker tag SRC TARGET                docker rmi IMAGE        docker history IMAGE
# Networks and volumes
docker network ls|create|inspect|rm|connect|disconnect
docker volume ls|create|inspect|rm
docker run -v NAME:/path IMAGE          # named volume
docker run -v /host/path:/path IMAGE     # bind mount
docker run --mount type=volume,source=NAME,target=/path IMAGE
# Cleanup
docker system df    docker system prune [-a] [--volumes]
docker container|image|volume|network prune
# Compose
docker compose up [-d] [--build]    docker compose down [--volumes]
docker compose ps|logs [-f]|exec|build|stop|start|restart|config
# Swarm
docker swarm init|join|leave    docker node ls|inspect|update
docker service create|ls|ps|update|scale|logs
docker stack deploy -c FILE STACK    docker stack ls|services|ps|rm
docker secret create|ls
# Kubernetes
kubectl get pods,deploy,svc,nodes    kubectl describe pod NAME
kubectl logs [-f] POD [-c CONTAINER]    kubectl exec -it POD -- sh
kubectl apply|delete -f FILE.yaml    kubectl scale deployment NAME --replicas=N
kubectl set image deployment/NAME CONTAINER=IMAGE
kubectl rollout status|history|undo deployment/NAME
kubectl get events --sort-by=.metadata.creationTimestamp

```

Appendix B: Dockerfile Instruction Reference

Instruction	Purpose	Example
FROM / LABEL	Base image / metadata	FROM node:20-alpine, LABEL version="1.0"
ARG / ENV	Build-time / runtime variable	ARG BUILD_ENV=production, ENV PORT=8080
WORKDIR	Working directory	WORKDIR /app
COPY / ADD	Copy files / copy + extract or fetch	COPY . ., ADD app.tar.gz /app
RUN	Build-time command	RUN npm install
EXPOSE / VOLUME	Document port / declare mount point	EXPOSE 8080, VOLUME /data
USER	User for later instructions	USER appuser
ENTRYPOINT / CMD	Fixed executable / default args	ENTRYPOINT ["python", "app.py"], CMD ["--port", "8080"]
HEALTHCHECK	Container health test	HEALTHCHECK CMD curl -f localhost/health

Instruction	Purpose	Example
STOPSIGNAL / ONBUILD	Stop signal / deferred instruction	STOPSIGNAL SIGTERM, ONBUILD COPY . /app
SHELL	Default shell for shell-form instructions	SHELL ["/bin/bash", "-c"]

Appendix C: Glossary

- **Bind mount** — mount of an exact host path into a container.
- **Bridge network** — default single-host isolated private network.
- **Build context** — files sent to the daemon for a build.
- **cgroups** — kernel feature limiting and accounting CPU, memory, I/O per process group.
- **CMD / ENTRYPOINT** — default command or arguments / fixed executable of a container.
- **Container** — running or stopped instance of an image, isolated by namespaces and cgroups.
- **containerd** — high-level runtime for image transfer, storage, and execution; used by Docker and Kubernetes.
- **Deployment** — Kubernetes resource managing replicated, self-healing Pods.
- **Digest** — SHA-256 hash identifying an image's exact content.
- **Dockerfile** — text file of image-build instructions.
- **Image** — immutable layered filesystem template for containers.
- **Ingress** — Kubernetes resource for HTTP(S) routing into the cluster.
- **Kubernetes (k8s)** — industry-standard orchestration platform.
- **Layer** — read-only slice of an image, typically one Dockerfile instruction.
- **Namespace (Linux / Kubernetes)** — isolated view of system resources per process / virtual sub-cluster.
- **OCI** — organization defining standard image and runtime specifications.
- **Orchestrator** — automates deployment, scaling, networking, and management across machines.
- **Overlay network** — multi-host network driver used by Swarm.
- **Pod** — smallest Kubernetes deployable unit; containers sharing network and storage.
- **Registry** — service that stores and distributes images (e.g., Docker Hub).
- **Rolling update** — incremental replacement of instances to avoid downtime.
- **runc** — low-level OCI runtime that creates the container process.
- **Service (Swarm/Kubernetes)** — abstraction over replicated instances with stable networking.
- **Twelve-Factor App** — SaaS methodology, including configuration in the environment.
- **Volume** — Docker-managed persistent storage independent of any container.
- **WSL2** — Windows Subsystem for Linux 2; real Linux kernel on Windows, Docker Desktop's default backend.

Appendix D: Review Questions and Answers

Self-check each chapter; answers follow each question.

D.1 Chapters 1-3

- **Which kernel features isolate containers?** Namespaces (PIDs, network, mounts, etc.) and cgroups (resource limits and accounting).
- **Why are container images MBs and VM images GBs?** Containers share the host kernel and package only the app and dependencies; VMs bundle a full guest OS.
- **Who standardized image and runtime formats, and why does it matter?** The OCI; it keeps Docker, containerd, Kubernetes, and Podman interoperable.
- **Two DevOps principles besides automation?** CI/CD and Infrastructure as Code (also monitoring, shared ownership).
- **Three components of Docker's architecture?** Client, daemon (**dockerd**), and registry.
- **How do dockerd, containerd, and runc relate?** dockerd delegates lifecycle to containerd, which uses runc (via a shim) to create the process with namespaces and cgroups.
- **Why can the writable layer be deleted safely?** It sits on read-only image layers via copy-on-write; image layers are never modified.
- **Why does Docker Desktop on macOS run a Linux VM?** Containers need a Linux kernel and macOS's XNU cannot run them.
- **Best Windows backend and why?** WSL2: a real lightweight Linux kernel with near-native performance, replacing legacy Hyper-V.
- **Why avoid sudo day to day?** Add your user to the docker group instead, remembering that membership is effectively root.

D.2 Chapters 4-8

- **stop vs. rm?** stop halts but keeps the container; rm deletes it.
- **run vs. exec?** run creates a new container; exec runs an extra process in a running one.
- **Why does docker run ubuntu bash exit immediately?** Without **-it** there is no TTY or STDIN, so the shell exits.
- **ENV default vs. -e?** The runtime **-e** value wins.
- **Why not bake secrets in via ENV?** They become part of the image, visible to anyone who can pull or inspect it.
- **Why copy package.json and install before copying source?** A step is cached until its inputs change; the slow, rarely changing install stays cached.
- **Key advantage of multi-stage builds?** The final image holds only runtime artifacts, cutting size and attack surface.

- **With `ENTRYPOINT ["python", "app.py"]` and `CMD ["--debug"]`, what does `docker run myimage --port 9000` run?** `python app.py --port 9000`; runtime arguments replace `CMD` but append to `ENTRYPOINT`.
- **Why can't containers on the default bridge resolve names?** It has no embedded DNS; only user-defined bridges do.
- **Does `EXPOSE 8080` publish the port?** No, it is documentation; use `-p/-P` or `Compose ports:`.
- **When use a bind mount over a named volume?** When you need a specific existing host path, mainly live-editing source in development.
- **What happens to `tmpfs` data when the container stops?** It is gone; `tmpfs` lives only in RAM.

D.3 Chapters 9-13

- **Does `depends_on` guarantee readiness?** No, only start order; use `condition: service_healthy` with a healthcheck.
- **How to bring an app down including named volumes?** `docker compose down --volumes`; plain `down` keeps them.
- **Why deploy by digest rather than tag?** Tags are mutable; a digest immutably identifies exact content.
- **Which kernel feature underlies `overlay2`?** `OverlayFS`, the layered copy-on-write filesystem.
- **A benefit of `BuildKit`?** Parallel stages, better cache import/export, or build secrets that never enter layers.
- **Why is bind-mount I/O slower on Docker Desktop?** Files cross the boundary between the host OS and the Linux VM/WSL2.
- **What is desired-state reconciliation?** You declare the target (e.g., 5 replicas) and the orchestrator continuously corrects drift.

D.4 Chapters 14-17

- **Why an odd number of Swarm managers?** Raft needs a majority quorum; odd counts avoid ties and maximize fault tolerance.
- **What does the routing mesh do?** Makes a published port reachable on every node, routing traffic to a node running a replica.
- **Smallest Kubernetes unit?** The Pod: containers scheduled together sharing a network namespace and optionally volumes.
- **Readiness vs. liveness?** Failed readiness removes the Pod from load balancing without killing it; failed liveness makes the kubelet restart it.
- **Why doesn't modern Kubernetes use the Docker Engine?** `Dockershim` was removed in 1.24+; it uses `containerd` or `CRI-O` via `CRI`, and Docker images remain compatible as OCI images.

- **Why tag CI images with the commit SHA?** Each build gets a unique, immutable, traceable identifier back to its source commit.
- **Canary vs. rolling update?** Canary routes a small, controlled traffic share to the new version first; rolling simply replaces instances incrementally.
- **Why avoid `--privileged`?** It grants nearly all capabilities and device access, removing container isolation.
- **Risk of deleting a secret file in a later instruction?** It remains in the earlier layer and can be extracted from image history.

Appendix E: Further Resources

- **Documentation:** docs.docker.com (CLI, Dockerfile, Compose), kubernetes.io/docs (with interactive tutorials), opencontainers.org (specs from Ch. 11).
- **Hands-on practice:** Play with Docker (labs.play-with-docker.com); browser-based Kubernetes scenarios (Killercodea-style); kind, minikube, and k3d for local clusters.
- **Reference material:** Docker Hub official images (study the Dockerfiles of postgres, nginx, node) and 12factor.net.
- **Security:** CIS Docker and Kubernetes Benchmarks; Trivy, Docker Scout, and Gripe scanners; Falco for runtime monitoring.
- **Suggested practice progression:** (1) containerize a small project; (2) add a database via Compose; (3) push to a registry and pull on another machine; (4) build a CI pipeline; (5) deploy to a local Kubernetes cluster; (6) add probes and break the app to watch self-healing; (7) add resource limits and load-test them.